Rota Vital — Trabalho U2 · Parte 2: Planejamento da remodelagem

Squad Hemotrack · Infraestrutura de Software · ADS · CESAR School · novembro de 2026

Pré-entrega (versão parcial, 17/11): seções A e C

Identificação e premissas

O squad Hemotrack remodela o Rota Vital como um monolito modular em três camadas com cinco bounded contexts, projeta quatro microsserviços e orça três cenários entre US$ 47 e US$ 470 por mês.

Item	Preencher
Squad	Hemotrack
Integrantes	Ewerton Guilherme da Silva (Product Owner) · Pablo Arthur Eustáquio de Lima, João Ricardo Alves de Brito, Lucas Aprigio dos Santos, Saulo Eduardo Almeida dos Santos (Backend) · Thiago Cardozo da Conceição, Eloi de Lima Sousa (Frontend / UI Design)
Repositório	github.com/egs3-coder/Trabalho-U2-Parte-2-Planejamento-da-remodelagem — pasta `docs/arquitetura/`
Disciplina	Infraestrutura de Software · ADS · CESAR School · 2026.2

Premissas: o Rota Vital coordena bolsas de sangue entre um hemocentro e hospitais: requisição → reserva no estoque → expedição em caixa térmica com sensor → transporte → recebimento. Stack da Parte 1: Java 21 + Spring Boot, PostgreSQL, Docker e GitHub Actions. Apenas dados sintéticos.

A. Remodelagem do domínio com DDD

O coração do Rota Vital é atender a requisição com a bolsa certa, dentro da validade e sem quebrar a cadeia fria; o resto apoia isso ou é comprável pronto.

A.1 Subdomínios

Subdomínio	Tipo	Justificativa
Atendimento de requisições	Core	Escolher bolsas compatíveis e mais próximas do vencimento, por prioridade, é o diferencial do produto.
Rastreabilidade da cadeia fria	Core	Provar que a bolsa viajou na faixa de temperatura é o que gera confiança no hemocentro e no hospital.
Gestão de estoque	Suporte	Específico do negócio, mas é controle de entrada, saída e validade.
Expedição e documentação	Suporte	Obrigatório, porém segue um procedimento padronizado.
Cadastro da rede	Suporte	Dado de referência com regras simples.
Identidade e acesso	Genérico	Resolvido por produto pronto (Keycloak, Cognito).
Notificações	Genérico	Envio de e-mail e SMS é commodity, contratado de um provedor.

A.2 Bounded contexts

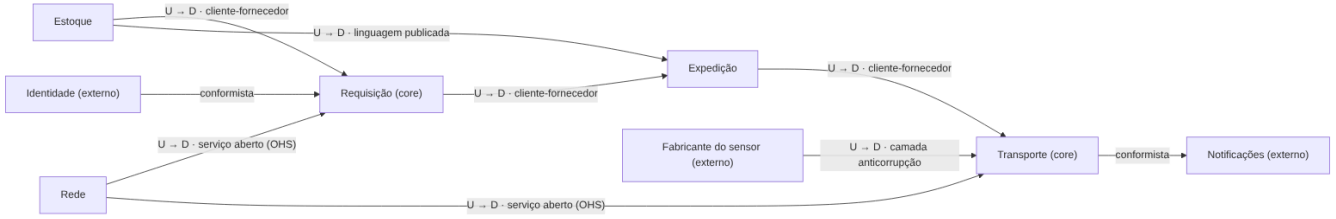
Contexto	Responsabilidade	Subdomínio
Estoque	Registra bolsas, controla validade e status e reserva bolsas para requisições.	Gestão de estoque
Requisição	Recebe o pedido, aplica prioridade e compatibilidade e acompanha até o recebimento.	Atendimento (core)
Expedição	Separa as bolsas reservadas, acondiciona em caixa térmica e emite a guia.	Expedição
Transporte	Acompanha a viagem, recebe a telemetria e detecta excursões de temperatura.	Cadeia fria (core)
Rede	Mantém hospitais, unidades solicitantes, endereços e contatos.	Cadastro da rede

A.3 Glossário da linguagem ubíqua

Termo	Contexto	Significado
Bolsa	Estoque	Unidade de hemocomponente com número de doação, tipo sanguíneo, validade e status.
Bolsa	Expedição	Volume a acondicionar: só importam código de barras, posição na caixa e conferência.
Entrega	Requisição	Confirmação do hospital de que recebeu o pedido; encerra a requisição.

Termo	Contexto	Significado
Entrega	Transporte	Chegada da caixa ao destino, com horário e última temperatura; encerra a viagem.
Tipo sanguíneo	Estoque, Requisição	Grupo ABO + fator Rh (ex.: O−).
Validade	Estoque	Data-limite de uso; bolsa vencida não pode ser reservada.
Reserva	Estoque	Bloqueio temporário de bolsas para uma requisição, com prazo de expiração.
Requisição	Requisição	Pedido de um hospital com itens, prioridade e justificativa clínica.
Prioridade	Requisição	Rotina, urgência ou emergência; define o prazo e a ordem na fila.
Guia de remessa	Expedição	Documento com as bolsas, origem, destino e responsável.
Leitura de telemetria	Transporte	Temperatura, hora e posição enviadas pelo sensor da caixa.
Excursão de temperatura	Transporte	Período fora da faixa permitida (ex.: 2–6 °C para hemácias).

A.4 Mapa de contextos



Relação (upstream → downstream)	Tipo	Como funciona
Estoque → Requisição	Cliente-fornecedor	Requisição pede reservas; Estoque mantém a API `reservar` e o evento `BolsasReservadas`.
Requisição → Expedição	Cliente-fornecedor	Expedição começa ao receber `RequisicaoAtendida`.
Estoque → Expedição	Linguagem publicada	Código ISBT 128 da bolsa como identificador comum.
Expedição → Transporte	Cliente-fornecedor	Transporte abre a viagem a partir de `RemessaDespachada`.
Sensor → Transporte	Camada anticorrupção	Adaptador traduz o payload do fabricante (°F) para `LeituraTelemetria` (°C).
Rede → Requisição e Transporte	Serviço aberto	API de consulta de unidades e endereços.
Identidade, Notificações	Conformista	O Rota Vital adota o modelo do provedor sem traduzir.

Eventos de domínio (opcional): `RequisicaoRegistrada` e `RequisicaoAtendida` (Requisição), `BolsasReservadas` e `BolsaVencida` (Estoque), `RemessaDespachada` (Expedição), `TemperaturaForaDaFaixa` e `EntregaConcluida` (Transporte); cada um é consumido pelo contexto seguinte do fluxo.

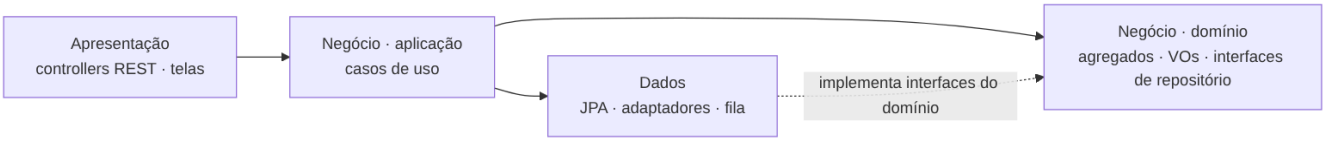
C. Arquitetura em três camadas

Cada bounded context é um módulo com as mesmas três camadas; as dependências apontam para baixo e o domínio não importa Spring, JPA nem HTTP.

C.1 O que fica em cada camada

Camada	Contém	Exemplos
Apresentação	Controllers REST, DTOs, validação de formato, telas web e app do motorista	`RequisicaoController`, `AlocarBolsaRequest`
Negócio · aplicação	Casos de uso: orquestram o domínio, abrem a transação, publicam eventos	`RegistrarRequisicao`, `AlocarBolsa`
Negócio · domínio	Agregados, entidades, objetos de valor, eventos e interfaces de repositório	`Requisicao`, `TipoSanguineo`, `RequisicaoRepository`
Dados	Repositórios JPA, banco, integrações (sensor, notificações) e fila	`RequisicaoRepositoryJpa`, `SensorTelemetriaAdapter`

C.2 Regra de dependência



- Cada camada só usa a de baixo: apresentação → aplicação → domínio e dados.
- O domínio é Java puro: a interface `RequisicaoRepository` fica no domínio e a implementação JPA fica em dados (inversão de dependência).
- Um contexto só fala com outro pela camada de aplicação ou por evento. Testes **ArchUnit** na esteira falham se `..dominio..` importar Spring, JPA ou `..apresentacao...`

C.3 Estrutura de pacotes

```
br.cesar.rotavital
├── estoque
│   ├── apresentacao   (EstoqueController)
│   ├── negocio
│   │   ├── aplicacao  (RegistrarBolsa, ConsultarEstoque, ReservarBolsas)
│   │   └── dominio    (Bolsa, Validade, Reserva, BolsaVencida, BolsaRepository)
│   └── dados          (BolsaRepositoryJpa)
├── requisicao
│   ├── apresentacao   (RequisicaoController)
│   ├── negocio
│   │   ├── aplicacao  (RegistrarRequisicao, AlocarBolsa, MarcarRequisicaoAtendida)
│   │   └── dominio    (Requisicao, ItemRequisicao, AlocacaoBolsa, Prioridade)
│   └── dados          (RequisicaoRepositoryJpa, EstoqueClient)
├── expedicao          apresentacao · negocio/aplicacao (MontarRemessa, EmitirGuia)
│                      · negocio/dominio (Remessa, CaixaTermica, GuiaRemessa) · dados
├── transporte         apresentacao · negocio/aplicacao (IniciarViagem, RegistrarLeitura)
│                      · negocio/dominio (Viagem, LeituraTelemetria) · dados (SensorTelemetriaAdapter)
└── rede               apresentacao · negocio/aplicacao (CadastrarUnidade)
                       · negocio/dominio (UnidadeSolicitante, Endereco) · dados
```

C.4 Camada lógica × tier físico

Camada é organização do código; tier é onde o processo roda. As três camadas do servidor rodam no **mesmo** processo; a camada de dados (código dos repositórios) roda no tier de aplicação, e o tier de dados é o servidor de banco.

Tier físico	O que roda	Camadas lógicas
Cliente	Navegador (painéis do hemocentro e do hospital) e app do motorista	Telas da apresentação
Aplicação	Container Docker com o JAR Spring Boot (Render ou ECS Fargate)	Apresentação, negócio e dados
Dados	PostgreSQL gerenciado e, nos cenários maiores, a fila	Persistência